

## Performance Testing

|                      |                                                    |
|----------------------|----------------------------------------------------|
| <b>Date</b>          |                                                    |
| <b>Team ID</b>       | xxxxxx                                             |
| <b>Project Name</b>  | EduGenie: Google Gemini Powered Learning Assistant |
| <b>Maximum Marks</b> | 5 Marks                                            |

### Performance Testing

Performance testing evaluates how EduGenie behaves under expected and peak load. Tests were run with the project's own asynchronous load-test script (EduGenie/tests/load\_test.py).

#### Step 1: Testing Overview

| Field                      | Details                                                                                                                                                                                                                        |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Testing Tool Used</b>   | Custom Python load tester (tests/load_test.py, asyncio + httpx); pytest for functional tests; psutil for CPU/memory                                                                                                            |
| <b>Type of Testing</b>     | Load testing (10 concurrent users) and stress testing (50 concurrent users)                                                                                                                                                    |
| <b>Target Module / API</b> | GET /, GET /health, GET /qa, POST /explain, POST /quiz, POST /summarize, GET /learn/recommendations                                                                                                                            |
| <b>Test Environment</b>    | Local - Uvicorn (1 worker) on a 4-core / 15.7 GB Linux machine. The Gemini call was replaced by an instant stub to measure EduGenie's own server overhead (end-to-end AI latency depends on Google's API and the key's quota). |
| <b>Test Date</b>           |                                                                                                                                                                                                                                |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description                                                                  | No. of Virtual Users | Duration (sec) | Expected Outcome                                               |
|------|----------------------------------------------------------------------------------------------|----------------------|----------------|----------------------------------------------------------------|
| 1    | Home page and health check load (GET /, GET /health)                                         | 10                   | 10 each        | Avg < 2 s, 0% errors                                           |
| 2    | All five AI endpoints under normal load (Q&A, explain, quiz, summarize, learning path)       | 10                   | 10 each        | Avg < 2 s (server overhead), 0% errors                         |
| 3    | Stress test on the most-used endpoints (Q&A and quiz)                                        | 50                   | 10 each        | No failures, max < 5 s                                         |
| 4    | End-to-end test with the live Gemini API (python tests/load_test.py --users 5 --duration 30) | 5                    | 30             | AI responses within ~10 s; limited by the API key's rate limit |

### Step 3: Performance Test Results

| S.No | Metric                               | Target Value | Actual Value                             | Status (Pass / Fail) | Remarks                                         |
|------|--------------------------------------|--------------|------------------------------------------|----------------------|-------------------------------------------------|
| 1    | Response Time (Avg)                  | < 2 seconds  | 11-12 ms (10 users); 51-58 ms (50 users) | Pass                 | Server overhead only (Gemini stubbed)           |
| 2    | Response Time (Max)                  | < 5 seconds  | 218 ms                                   | Pass                 | Worst case across all scenarios                 |
| 3    | Throughput (Req/sec)                 | -            | 824-985 req/s                            | Pass                 | Single Uvicorn worker                           |
| 4    | Error Rate                           | < 1%         | 0.0%                                     | Pass                 | ~77,000 requests, no failures                   |
| 5    | CPU Utilization                      | < 80%        | 14.5% peak (of 4 cores)                  | Pass                 | Measured with psutil during load                |
| 6    | Memory Utilization                   | < 80%        | 54 MB (0.3% of RAM)                      | Pass                 | Lightweight - no local model loaded             |
| 7    | End-to-end AI response (live Gemini) | < 10 seconds |                                          |                      | To be measured with a live API key (scenario 4) |

### Step 4: Observations & Analysis

- EduGenie's own processing (routing, validation, prompt building, JSON parsing, response) adds only ~12 ms on average, so total response time is dominated by the Gemini API.
- No errors occurred at 50 concurrent users; blocking Gemini calls run in FastAPI's thread pool, keeping the server responsive.
- Memory use stays very low (~54 MB) because AI inference runs in Google Cloud; enabling the optional local LaMini-Flan-T5 model adds roughly 3 GB.
- For production scale, run several Uvicorn workers or containers behind a load balancer; the stateless design makes horizontal scaling straightforward.
- Gemini free-tier rate limits (HTTP 429) are the most likely bottleneck under real load; EduGenie reports these clearly to the user.

### Step 5: Screenshots / Evidence

```
(.env) EduGenie> python tests/load_test.py --users 10 --duration 10 (Gemini stubbed)
EduGenie load test against http://127.0.0.1:8766
Scenario  Users  Requests  Avg ms  p95 ms  Max ms  Req/s  Errors
-----
home      10      8394      11.9    42.2    182.9   838.7   0.0%
health    10      8995      11.1    37.7    166.8   899.0   0.0%
qa        10      8871      11.3    38.6    143.4   886.5   0.0%
explain   10      8245      12.1    42.4    201.1   824.0   0.0%
quiz      10      8345      12.0    37.9    145.1   833.9   0.0%
summarize 10      8438      11.9    42.7    217.6   843.2   0.0%
path      10      8878      11.3    38.6    163.0   887.5   0.0%

(.env) EduGenie> python tests/load_test.py --users 50 --duration 10 --scenarios qa,quiz
EduGenie load test against http://127.0.0.1:8766
Scenario  Users  Requests  Avg ms  p95 ms  Max ms  Req/s  Errors
-----
qa         50     9872      50.6    73.0    107.8   984.8   0.0%
quiz       50     8659      57.8    70.3    104.0   863.2   0.0%

Server resources during load: cpu_avg=10.1% cpu_max=14.5% (of all 4 cores; single-core max 58%) mem_max=54MB (0.3% of 15.7GB)
```

*Load test output (tests/load\_test.py) with server resource usage*

```
(.venv) EduGenie> pytest -v
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /home/user/EduGenie-AI/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/user/EduGenie-AI/EduGenie
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collecting ... collected 34 items

tests/test_api.py::test_index_page_renders PASSED [ 2%]
tests/test_api.py::test_static_files_served PASSED [ 5%]
tests/test_api.py::test_health PASSED [ 8%]
tests/test_api.py::test_qa PASSED [ 11%]
tests/test_api.py::test_qa_requires_question PASSED [ 14%]
tests/test_api.py::test_explain PASSED [ 17%]
tests/test_api.py::test_explain_trailing_slash_like_original_doc PASSED [ 20%]
tests/test_api.py::test_explain_empty_topic PASSED [ 23%]
tests/test_api.py::test_explain_topic_too_long PASSED [ 26%]
tests/test_api.py::test_summarize PASSED [ 29%]
tests/test_api.py::test_summarize_empty PASSED [ 32%]
tests/test_api.py::test_quiz PASSED [ 35%]
tests/test_api.py::test_quiz_num_questions PASSED [ 38%]
tests/test_api.py::test_quiz_invalid_num_questions PASSED [ 41%]
tests/test_api.py::test_quiz_invalid_ai_output PASSED [ 44%]
tests/test_api.py::test_quiz_empty PASSED [ 47%]
tests/test_api.py::test_learning_recommendations PASSED [ 50%]
tests/test_api.py::test_learning_recommendations_requires_topic PASSED [ 52%]
tests/test_api.py::test_gemini_failure_returns_502 PASSED [ 55%]
tests/test_api.py::test_missing_api_key_returns_503 PASSED [ 58%]
tests/test_live_gemini.py::test_live_qa SKIPPED (set RUN LIVE TESTS=...) [ 61%]
tests/test_live_gemini.py::test_live_quiz SKIPPED (set RUN LIVE TESTS=...) [ 64%]
tests/test_live_gemini.py::test_live_learning_path SKIPPED (set RUN LIVE TESTS=...) [ 67%]
tests/test_modules.py::test_clean_json_block_removes_fences PASSED [ 70%]
tests/test_modules.py::test_parse_quiz_accepts_wrapped_object_and_skips_bad_items PASSED [ 73%]
tests/test_modules.py::test_parse_quiz_extract_array_from_surrounding_text PASSED [ 76%]
tests/test_modules.py::test_parse_quiz_raises_on_garbage PASSED [ 79%]
tests/test_modules.py::test_quiz_format_error_is_gemini_error PASSED [ 82%]
tests/test_modules.py::test_learning_path_invalid_level_defaults_to_beginner PASSED [ 85%]
tests/test_modules.py::test_explain_falls_back_to_gemini_when_local_fails PASSED [ 88%]
tests/test_modules.py::test_explain_uses_local_model_when_selected PASSED [ 91%]
tests/test_modules.py::test_generate_without_key_raises PASSED [ 94%]
tests/test_modules.py::test_placeholder_key_is_treated_as_missing PASSED [ 97%]
tests/test_modules.py::test_model_name_default_and_override PASSED [100%]

===== 31 passed, 3 skipped in 0.18s =====
```

*Functional test suite: 31 passed, 3 live tests skipped (require API key)*